UNIT 4: ANNS & DEEP LEARNING

1. ARTIFICIAL NEURAL NETWORKS (ANN)

Concept: Computational models inspired by the human brain. They consist of interconnected processing units (neurons) that map input data to output data by learning optimal weights.

THE PERCEPTRON (SIMPLEST ANN)

A single-layer neural network that takes a vector of real-valued inputs, calculates a linear combination of these inputs, and outputs a 1 or -1 using a step activation function.

- **Inputs (x_i):** The features of the data.
- **Weights (w_i):** The learned importance of each input.
- **Bias (w_0):** A threshold that shifts the activation function (always has input $x_0 = 1$).
- **Output:** $o(x) = sign(w_0 + w_1x_1 + \dots + w_nx_n)$

Limitation: A single perceptron can only learn **linearly separable** functions (like AND, OR). It completely fails at non-linear problems like XOR.

2. GRADIENT DESCENT & DELTA RULE

GRADIENT DESCENT

Technical Concept: An optimization algorithm used to minimize the network's Error (Loss) by taking steps in the opposite direction of the gradient of the error function with respect to the weights.

Imagine a blindfolded person trying to reach the bottom of a valley. They feel the slope of the ground (the gradient) and take a step downwards. The size of the step is controlled by the `Learning Rate (η)`.

THE DELTA RULE (WIDROW-HOFF RULE)

Used to update weights in an unthresholded perceptron (linear unit) by minimizing the squared error. It ensures the weights converge to a best-fit approximation even if the data is not perfectly linearly separable.

$$\Delta w_i = \eta \times (t - o) \times x_i$$
$$w_i(new) = w_i(old) + \Delta w_i$$

Where: η = learning rate, t = target (true) output, o = actual observed output, x_i = input value for that weight.

3. MULTILAYER PERCEPTRON (MLP)

Concept: A feedforward ANN with at least one **Hidden Layer** between the input and output layers. MLPs utilize non-linear activation functions (like Sigmoid, Tanh, or ReLU) instead of simple step functions.

- **Solving XOR:** By combining multiple linear boundaries in the hidden layer, MLPs can draw complex, non-linear decision boundaries, solving problems a single perceptron cannot.
- **Universal Approximation Theorem:** An MLP with just a single hidden layer (with enough neurons) can approximate any continuous mathematical function to any desired degree of accuracy.

4. DERIVATION OF BACKPROPAGATION

Backpropagation (6-Marker Detail): The algorithm used to train MLPs. It applies the *Chain Rule of Calculus* to compute the gradient of the loss function with respect to every single weight in the network, moving backward from the output layer to the input layer.

PHASE 1: FORWARD PASS

Input data is fed into the network. It passes through hidden layers, gets multiplied by current weights, passes through activation functions (e.g., Sigmoid: $\sigma(x) = 1 / (1 + e^{-x})$), and produces an output o_k.

Calculate Total Error using Mean Squared Error: $E = \frac{1}{2} \sum (t_k - o_k)^2$

PHASE 2: BACKWARD PASS (THE DERIVATION LOGIC)

We need to find how a change in a specific weight w_{ij} affects the total Error E. We calculate the partial derivative: $\partial E / \partial w_{ij}$.

Using the Chain Rule, we split this into three parts:

$$\partial E / \partial w_{ij} = (\partial E / \partial o_j) \times (\partial o_j / \partial net_j) \times (\partial net_j / \partial w_{ij})$$

1. $\partial E / \partial o_j$: How much the error changes if the neuron's output changes. (Calculated directly at output, or recursively for hidden layers).
2. $\partial o_j / \partial net_j$: The derivative of the activation function (For sigmoid, this is $o_j(1 - o_j)$).
3. $\partial net_j / \partial w_{ij}$: How the sum changes as the weight changes. This simply equals the input to that connection, x_i.

5. GENERALIZATION IN NEURAL NETWORKS

Technical Concept: Generalization is the network's ability to perform accurately on new, unseen data, not just the data it was trained on. Poor generalization means the network has **overfitted**.

PREVENTING OVERFITTING IN ANNS:

- `Early Stopping`: Monitor validation error during training. Stop training the moment validation error begins to rise, even if training error is still dropping.
- `Weight Decay (L2 Regularization)`: Add a penalty term to the error function that punishes large weights. This forces the network to use smaller, more distributed weights, creating a smoother function.
- **Dropout:** Randomly turn off a percentage of neurons during each training pass to prevent neurons from co-adapting too closely.

6. UNSUPERVISED LEARNING: SOM ALGORITHM

Self-Organizing Maps (SOM / Kohonen Maps): An unsupervised neural network used for dimensionality reduction and clustering. Unlike MLPs, SOMs use `Competitive Learning` instead of error-correction (backpropagation).

The Grid: Neurons are arranged in a 2D topographical grid. The goal is for the grid to "fold" itself to match the shape of the high-dimensional input data.

SOM ALGORITHM STEPS (EXAM FRAMEWORK)

1. **Initialize Weights:** Assign random small weights to all nodes in the 2D grid.
2. **Input:** Present a random input vector from the training data.
3. **Competition:** Calculate the Euclidean distance between the input vector and all node weight vectors. The node with the shortest distance wins and becomes the **Best Matching Unit (BMU)**.
4. **Cooperation (Neighborhood):** Determine the topological neighborhood around the BMU. Nodes close to the BMU on the grid will also learn.
5. **Adaptation:** Pull the weights of the BMU and its neighbors closer to the input vector. Neighbors closer to the BMU are pulled stronger than distant neighbors.
6. **Repeat:** Shrink the learning rate and neighborhood radius over time, repeating for many epochs until the map stabilizes.

DEEP LEARNING

7. INTRODUCTION TO DEEP LEARNING

Definition: A subfield of machine learning based on ANNs with multiple hidden layers (hence "deep").

The Core Difference: In traditional ML (like Random Forests or standard ANNs), a human expert must manually extract features (e.g., finding edges in an image). In Deep Learning, the network performs **Automatic Feature Extraction**. The raw data (pixels) is fed in, and the network itself discovers the optimal features across its deep layers.

8. CONVOLUTIONAL NEURAL NETWORKS (CNN)

Concept: Specialized deep neural networks designed primarily to process data that has a known grid-like topology (e.g., 2D image pixels, 1D time-series data). They are heavily inspired by the human visual cortex.

CORE PROPERTIES OF CNNS:

- **Local Receptive Fields:** Neurons only connect to a small region of the input, not the whole image, preserving spatial relationships.
- **Shared Weights (Filters):** A single filter (e.g., an edge detector) sweeps across the entire image. This drastically reduces the number of parameters compared to a standard MLP.
- **Translation Invariance:** Because filters slide across the image, a CNN can recognize an object (like a cat) regardless of where it appears in the frame.

1D CONVOLUTION

Data Type: Sequential data, Time-series, Audio signals, Text data.

How it works: The filter (kernel) slides in only **one dimension** (e.g., left to right across time). It looks at sequences of data points to find temporal patterns.

2D CONVOLUTION

Data Type: Images, Spatial data.

How it works: The filter is a 2D matrix (e.g., 3x3) that slides in **two dimensions** (width and height) across the image matrix, computing dot products to detect spatial features like edges and corners.

9. TYPES OF LAYERS IN A CNN

1. CONVOLUTIONAL LAYER

The core building block. It applies multiple learnable filters (kernels) to the input. Each filter slides across the image, performing matrix multiplication, to output a 2D **Feature Map**. Early layers learn simple edges; deep layers learn complex objects (like faces).

2. ACTIVATION FUNCTION LAYER

Applies a non-linear mathematical operation to the output of the convolution. The standard is **ReLU (Rectified Linear Unit)**: $f(x) = max(0, x)$. It replaces all negative pixel values with zero, introducing non-linearity without affecting receptive fields.

3. POOLING LAYER (DOWNSAMPLING)

Reduces the spatial dimensions (width/height) of the feature maps, reducing computation and controlling overfitting.

- **Max Pooling:** Slides a window (e.g., 2x2) and outputs only the highest value in that window. (Most common).
- **Average Pooling:** Outputs the average of the values in the window.

4. FULLY CONNECTED (FC) LAYER

Placed at the very end of the network. The 2D feature maps are "flattened" into a 1D vector. This vector is fed into a standard Multilayer Perceptron (FC layer), which outputs the final classification probabilities (e.g., using Softmax).

10. TRAINING OF CNN NETWORKS

CNNs are trained using the same foundational principle as MLPs: **Backpropagation**.

- **Loss Function:** Calculate how far the prediction is from the truth (e.g., Categorical Cross-Entropy for multi-class image classification).
- **Backprop through Convolution:** The error gradient is passed back not just through weights, but through the pooling layers and convolution filters. The filters themselves update their matrix values to become better feature detectors.
- **Optimizers:** Standard Gradient Descent is too slow. CNNs typically use advanced adaptive optimizers like `Adam` or `RMSProp` to speed up convergence.

11. CASE STUDIES: APPLICATIONS OF DL & CNN

CASE 1: DIABETIC RETINOPATHY (MEDICAL IMAGING)

Problem: Identifying damage to the retina caused by diabetes, which can lead to blindness. Manually grading retina scans is slow and prone to human error.

CNN Solution: A 2D CNN is trained on thousands of high-resolution fundus images graded by expert ophthalmologists.

- **Architecture:** Deep networks like ResNet or Inception.
- **Features learned:** The CNN automatically learns to detect microaneurysms, hemorrhages, and hard exudates in the eye without being explicitly programmed to look for them.
- **Output:** A probability score ranking the severity of the disease (0 to 4 scale).

CASE 2: BUILDING A SMART SPEAKER (ALEXA / SIRI)

Problem: Recognizing spoken commands ("Wake words") in real-time, amidst background noise.

CNN Solution: Uses **1D CNNs** or Recurrent Neural Networks (RNNs).

- **Preprocessing:** The raw audio waveform is converted into a spectrogram (a visual representation of frequencies over time).
- **Processing:** A 1D or 2D CNN scans this spectrogram to identify the specific frequency patterns associated with phonemes and words.
- **Execution:** If the probability of the wake word exceeds a threshold, the speaker activates the main processor.

CASE 3: SELF-DRIVING CARS (AUTONOMOUS VEHICLES)

Problem: The car must understand its environment in real-time (lanes, pedestrians, signs) to navigate safely.

CNN Solution: Heavy reliance on real-time 2D CNNs processing video feeds from multiple cameras.

- **Object Detection (YOLO/SSD):** Bounding boxes are drawn around cars, pedestrians, and traffic lights.
- **Semantic Segmentation:** Assigning a class to *every single pixel* (e.g., this pixel is road, this pixel is sidewalk) so the car knows exactly where the drivable space is.